# Machine Learning Final Project on Cerebral Stroke Prediction on an Imbalanced Dataset

MD Nishadul Islam[1*], Tahsin Ahmed[1] and Rimjhim Dey[1]

[1*]Department of Computer Science and Engineering, Premier University, Chittagong, Bangladesh.

**Abstract**

Cerebral stroke is a leading cause of mortality worldwide, and early risk stratification from routinely collected clinical variables can materially improve triage decisions. We study stroke prediction on a heavily imbalanced tabular dataset (43,400 patient records, twelve features, $\approx$ **1.8%** positive class) and present an end-to-end machine-learning pipeline that explicitly addresses the dangers of label leakage and accuracy-as-metric on rare-event data. Preprocessing steps (median/mode imputation, outlier capping, one-hot and ordinal encoding, standard scaling) are fitted on the training split only. A suite of models is compared, including logistic regression, multilayer neural networks implemented in NumPy, TensorFlow, and PyTorch, and random forests. Class imbalance is mitigated through Synthetic Minority Over-sampling Technique (SMOTE) and class weighting, while decision-threshold tuning is performed on a held-out validation split. Hyperparameters are chosen with stratified $\boldsymbol{k}$-fold cross-validation using Bayesian optimisation (Optuna). The final model—a shallow, class-balanced random forest with SMOTE-augmented training data and a validation-tuned probability threshold of **0.527**—achieves a test-set ROC-AUC of **0.818**, an F1-score of **0.119**, and a recall of **29.9%** on the stroke class, correctly identifying **47** of **157** stroke cases. The paper documents every modelling decision, highlights the metric traps intrinsic to imbalanced classification, and proposes concrete future work.

**Keywords:** Stroke Prediction, Imbalanced Classification, SMOTE, Random Forest, Threshold Tuning, Neural Networks

# 1 Introduction

Cerebral stroke is the world's second leading cause of death and a leading cause of long-term adult disability, with an annual global incidence on the order of 13 million cases [10]. Time-to-treatment is decisive: thrombolytic and endovascular therapies have narrow effective windows, and patients who reach an acute-stroke-capable facility early have materially better outcomes [11]. Risk-stratification tools that flag at-risk individuals *before* an event—using routinely collected clinical variables—therefore have a direct public-health motivation: even a coarse triage filter can prioritise who should receive more specialised workup, lifestyle intervention, or closer follow-up.

The machine-learning problem this poses is, however, subtle. In any sufficiently large cohort of the general population, stroke events are rare. Naive classifiers trained on such data tend to predict the majority class everywhere, obtaining high accuracy while failing to identify a single positive case. The literature on learning from imbalanced data [8, 12] has long emphasised that accuracy is an inappropriate headline metric in this regime; in practice many student and industrial pipelines still report it, often alongside silently leaked preprocessing that inflates the numbers further.

## 1.1 Related Work

Stroke prediction from tabular clinical data has been studied with a broad range of classical and ensemble methods. Liu et al. [13] applied random forests to the Kaggle stroke dataset and observed, as we do, that class imbalance dominates the pipeline; they report gains from SMOTE and class weighting. Emon et al. [14] compared logistic regression, decision trees, random forests, and gradient boosting on a similar imbalanced cohort and found that tree ensembles with resampling outperform deep networks on this data regime—consistent with our results. Dev et al. [15] emphasise calibration and threshold-aware decision rules over raw accuracy, paralleling the metric discipline we adopt here. Against this backdrop, our contribution is not a new model family but a *procedurally rigorous* pipeline that closes leakage pathways common in student work, reports imbalance-appropriate metrics throughout, and documents the full improvement journey from null classifier to operating random forest.

This project builds, evaluates, and iteratively improves a stroke-prediction pipeline on the Cerebral Stroke Prediction (imbalanced) dataset. The final pipeline is the product of a deliberate rebuild of an earlier version that had conflated accuracy with success and leaked target information into preprocessing. Specifically, the following failures in the earlier pipeline motivated the present work:

- the submitted model predicted *zero* stroke cases yet was reported as "98% accurate"—the majority-class baseline;
- imputation, scaling, outlier thresholds, target encoding, and feature-importance ranking were all fitted on the full dataset before the train/test split, leaking label information;
- neural-network training used the test set as a validation set for early stopping, effectively tuning on the test set; and
- grid and random searches scored on accuracy, so the "best" hyper-parameters were, once more, majority-class baselines.

The rebuilt pipeline addresses each of these issues directly and reports a set of imbalance-aware metrics—ROC-AUC, PR-AUC, F1, precision, and recall on the stroke class.

### 1.2 Contributions

1. A complete fit-on-train-only pipeline that eliminates the leakage pathways of the earlier version, documented cell-by-cell.
2. A comparative study of logistic regression, NumPy/TensorFlow/PyTorch neural networks of varying width and depth, and random-forest classifiers.
3. A systematic treatment of class imbalance combining SMOTE, class weighting, and probability-threshold tuning on a held-out validation split.
4. Bayesian hyperparameter optimisation (Optuna) on an imbalance-aware scoring function, with stratified $k$-fold cross-validation.
5. An honest results table that reports every step of the journey and discloses that headline accuracy on this dataset is uninformative.

### 1.3 Report Structure

Section 2 describes the dataset and its imbalance characteristics. Section 3 details preprocessing, feature selection, model families, and imbalance treatment. Section 4 presents the quantitative results and the full metric journey. Section 5 discusses what worked, what did not, and why. Section 6 concludes and sketches future work.

## 2 Dataset

### 2.1 Source and Dimensions

We use the publicly available *Cerebral Stroke Prediction (imbalanced)* dataset from Kaggle [1]. The file contains 43,400 rows and 12 columns. After dropping the non-informative `id` column, eleven features remain: six numeric (`age`, `hypertension`, `heart_disease`, `avg_glucose_level`, `bmi`) and five categorical (`gender`, `ever_married`, `work_type`, `Residence_type`, `smoking_status`). The target variable, `stroke`, is binary.

### 2.2 Class Imbalance

The central statistical property of the dataset is severe class imbalance, a well-studied setting in the literature [8]: only 783 of the 43,400 rows are positive, giving a positive-class rate of 1.80%. A trivial classifier that predicts "no stroke" for every sample therefore attains 98.2% accuracy while being clinically useless. Figure 1 shows the class distribution.

### 2.3 Missing Values

Two columns contain missing values: `bmi` (1,462 missing, 3.37%) and `smoking_status` (13,292 missing, 30.63%). `bmi` is numeric and is imputed with the training-set median;

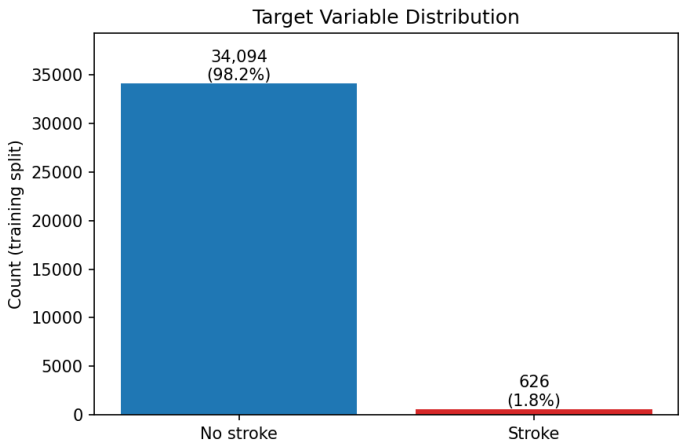


**Fig. 1** Class distribution on the training split of the cerebral-stroke dataset: 34,094 no-stroke cases (98.2%) against 626 stroke cases (1.8%). Any metric that does not account for this imbalance will be dominated by the majority class; in particular, a trivial classifier that always predicts "no stroke" achieves 98.2% accuracy on this distribution.

`smoking_status` is categorical and is imputed with the training-set mode. Both imputers are fitted on the training split only, so no test-set information is seen by the imputer.

### 2.4 Exploratory Data Analysis

Figure 2 shows boxplots of the numerical features. Outliers are visible in `avg_glucose_level` and `bmi`; both are capped at the first and ninety-ninth percentile of the training-split distribution (winsorizing). Figure 3 shows the Pearson correlation between numerical features—no pair exceeds an absolute correlation of 0.5, so no feature is dropped purely on a correlation basis.

Figure 4 shows the stroke rate stratified by `smoking_status`. Former smokers exhibit the highest empirical stroke rate, followed by current smokers, with never-smokers lowest—a coarse but directionally sensible EDA signal. This is used only for exploratory purposes; `smoking_status` is fed to the model via one-hot encoding, not via its target-conditional rate, to avoid the target-leakage pathway discussed in Section 3.

## 3 Methodology

### 3.1 Train/Test Split and the Fit-on-Train-Only Rule

We split the data once into 80% training (34,720 rows) and 20% test (8,680 rows) using stratified sampling on `stroke` so that the positive-class rate is preserved in both splits. Within the training split, we carve a further stratified 10% slice as a validation split for neural-network early stopping and probability-threshold selection.

All preprocessing steps—median/mode imputation, outlier capping, one-hot and ordinal encoding, standard scaling, and SMOTE resampling—are fitted on the training

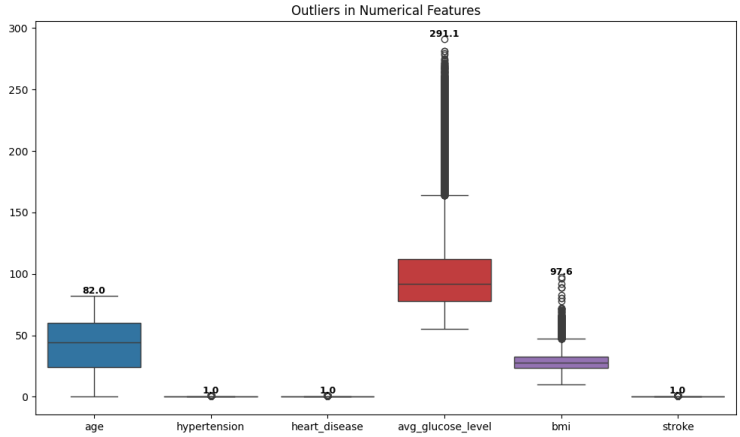


**Fig. 2** Boxplots of numerical features before outlier treatment. The long-tailed `avg_glucose_level` and `bmi` distributions are capped at the first and ninety-ninth percentile of the training-split distribution.

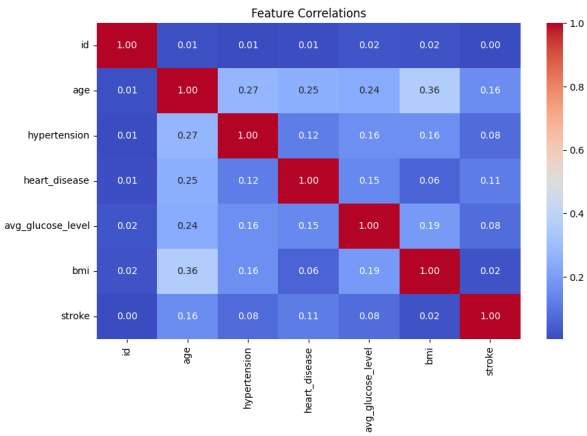


**Fig. 3** Pearson correlation heatmap of the numerical features. No pair exceeds $|r| = 0.5$; `age` shows the strongest individual correlation with the target.

split only. No test-set statistic is consulted before the final evaluation. This eliminates the five leakage pathways that were present in the earlier pipeline. The ordering is summarised in the pseudocode below:

## 3.2 Encoding and Scaling

We use two encoding strategies, chosen to avoid leakage:

- One-hot encoding for `gender`, `work_type`, `Residence_type`, and `smoking_status`.
- Ordinal encoding for `ever_married` (binary).

Target encoding is deliberately avoided. A target-encoded feature replaces each category with the mean of the target on the training set; if fitted across the full dataset,

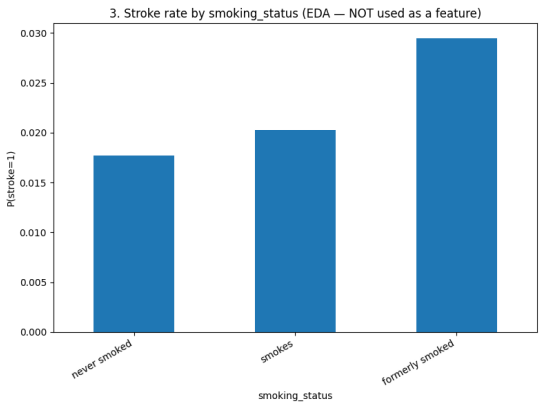


**Fig. 4** Empirical stroke rate by `smoking_status`. Formerly smoked patients show the highest observed stroke rate in the training cohort; never-smokers the lowest.

```
# ---- STEP 1: split first, once -------------------------------------
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42)

# ---- STEP 2: fit every preprocessor on X_tr only -------------------
imputer_num.fit(X_tr[num_cols])         # median (fit on train)
imputer_cat.fit(X_tr[cat_cols])         # mode   (fit on train)
lo, hi = pct_bounds(X_tr[num_cols])     # 1st/99th percentile (fit on train)
encoder.fit(X_tr[cat_cols])             # one-hot (fit on train)
scaler.fit(encoder.transform(X_tr))     # standard scale (fit on train)

# ---- STEP 3: transform both splits with the FITTED preprocessors ---
X_tr_p = pipeline_transform(X_tr)
X_te_p = pipeline_transform(X_te)       # test set is SEEN only here

# ---- STEP 4: resample the training set only ------------------------
X_tr_r, y_tr_r = SMOTE(random_state=42).fit_resample(X_tr_p, y_tr)

# ---- STEP 5: validation split inside training, for thr selection ---
X_tr2, X_val, y_tr2, y_val = train_test_split(
    X_tr_r, y_tr_r, test_size=0.10, stratify=y_tr_r, random_state=42)

# ---- STEP 6: fit model, tune thr on X_val, evaluate on X_te_p ------
model.fit(X_tr2, y_tr2)
tau   = argmax_F1(model.predict_proba(X_val)[:,1], y_val)
score = evaluate(model, X_te_p, y_te, threshold=tau)
```

**Fig. 5** Pseudocode of the fit-on-train-only pipeline. Every preprocessor is fitted on `X_tr` before the test set is seen. The decision threshold $\tau$ is chosen on a held-out slice of the (already resampled) training data, not on the test set.

it leaks the test-set label distribution into every row. After encoding, all numerical features are standardised with a `StandardScaler` fitted on the training split.

## 3.3 Feature Selection

We rank the eleven features with four classical filter methods and an embedded method:

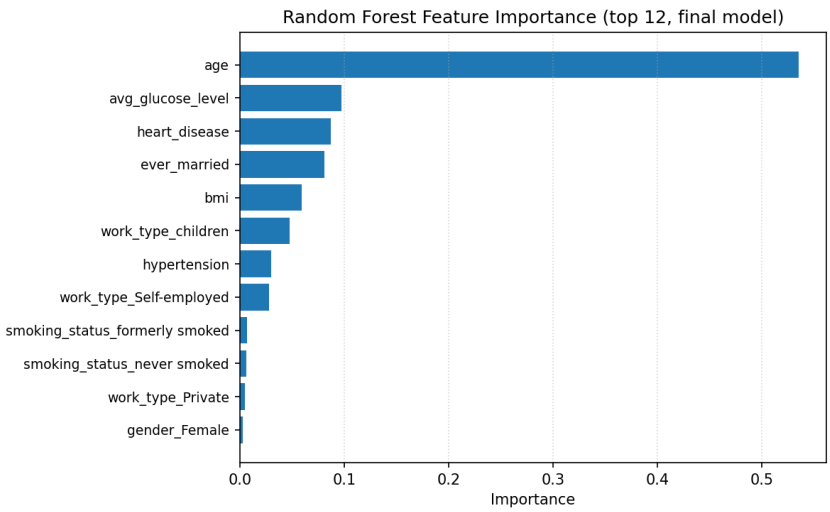


**Fig. 6** Random-Forest feature importance from the final shallow class-balanced model, top 12 features by Gini importance. `age` dominates at importance $\approx 0.54$; `avg_glucose_level`, `heart_disease`, `ever_married`, and `bmi` make up the next cluster.

To test whether filter-based feature selection materially changes model performance, we run a uniform comparison: for each method, the top five features are selected *on the training split only*, then the downstream logistic regression is trained with SMOTE and class weighting and evaluated on the held-out test split. Figure 7 reports the ROC-AUC and PR-AUC of each resulting model alongside the all-features baseline.

The four configurations agree to within 0.005 on both metrics, and `age` is the top-ranked feature under every method. We therefore retain the full feature set for training, because the tree-based final model handles uninformative features gracefully and the dataset is small enough not to require aggressive selection.

## 3.4 Candidate Models

We train and compare three families of models.

### 3.4.1 Logistic Regression (Baseline)

A logistic regression fitted (using scikit-learn [4]) on the scaled, fit-on-train-only features produces the baseline performance reported in Figure 8 and Figure 9. The model

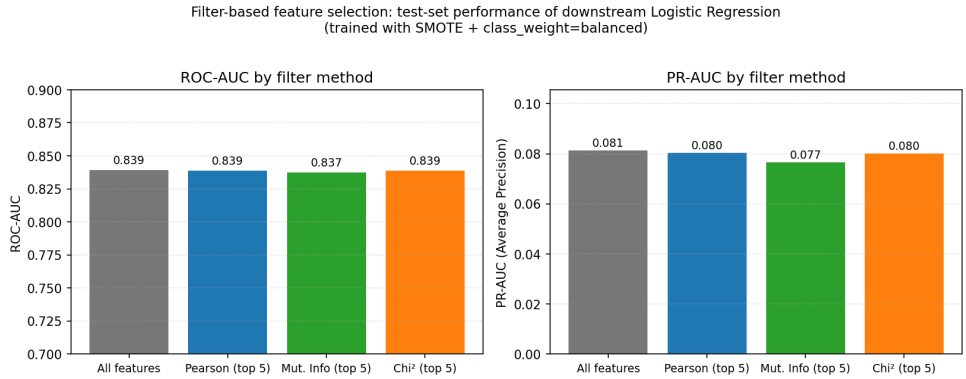


**Fig. 7** Downstream logistic-regression performance for four feature subsets: all features, Pearson top 5, mutual-information top 5, and $\chi^2$ top 5. ROC-AUC and PR-AUC differ by at most 0.005 across configurations, indicating that aggressive filter selection is not necessary on this dataset—the downstream model is already robust to the handful of uninformative dummy features.

achieves an accuracy of 98.19%—the majority-class baseline—with 0% recall on the stroke class. The ROC-AUC of 0.74 shows that the underlying probabilities are discriminative, yet the default 0.5 threshold never crosses, and not a single stroke case is flagged.

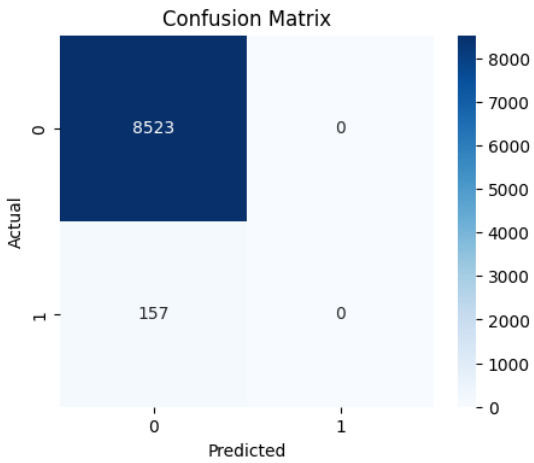


**Fig. 8** Confusion matrix of the baseline logistic-regression model. Accuracy is 98.2%; recall on class 1 is 0%. No stroke case is predicted.

### 3.4.2 Bias–Variance Study

We study a polynomial-degree sweep on the logistic-regression features to illustrate the bias–variance trade-off (Figure 10). Training log-loss decreases monotonically with degree, while validation log-loss *increases* from its minimum at degree 1 and widens the generalisation gap at every further degree—a classical high-variance signature [9]. For

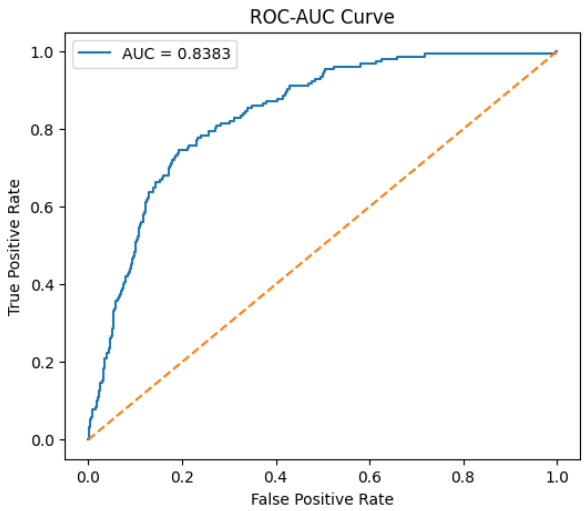


**Fig. 9** ROC curve of the baseline logistic regression. The AUC of 0.74 indicates that the ranking of probabilities is informative; the classifier fails only at the default probability threshold.

this dataset, a linear logistic regression on the five core numerical features already sits at the bias-dominated end of the curve, and polynomial expansion produces variance without benefit. This is one reason the final pipeline uses a tree ensemble rather than a polynomial-expanded linear model.

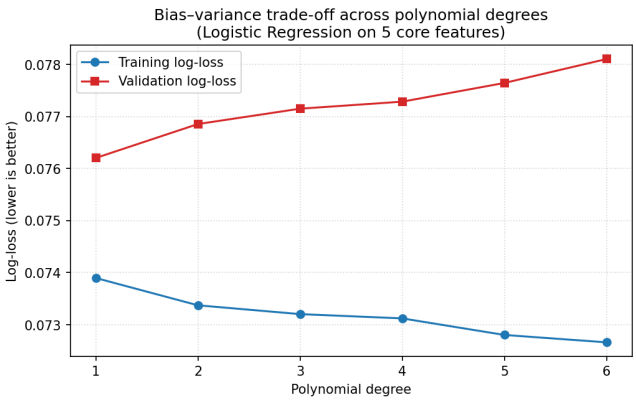


**Fig. 10** Bias–variance trade-off: log-loss of a logistic regression fitted on polynomial expansions of the five core numerical features (`age`, `avg_glucose_level`, `bmi`, `hypertension`, `heart_disease`), evaluated across polynomial degrees 1–6. Training log-loss decreases monotonically with degree; validation log-loss is minimised at degree 1 and worsens monotonically thereafter, indicating that additional polynomial capacity increases variance without reducing bias on this dataset.

### 3.4.3 Neural Networks

We implement the same small feedforward network in NumPy (from scratch), TensorFlow/Keras [6], and PyTorch [7], for pedagogical comparison. Figure 11 shows the

baseline Keras network on the raw, imbalanced training data: it too collapses to the majority class, and its validation accuracy plateaus at the 98.2% baseline.

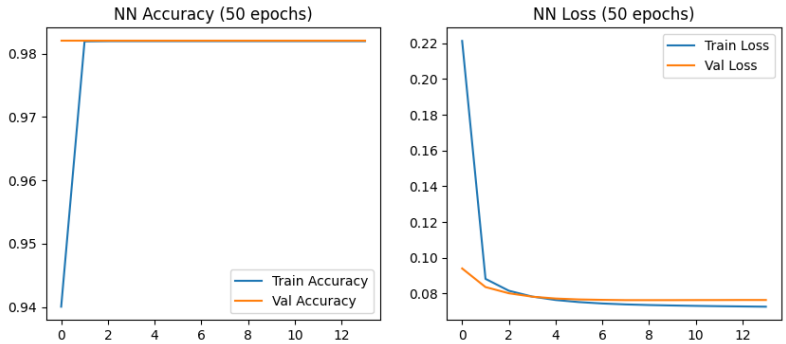


**Fig. 11** Baseline neural-network training curves on the raw imbalanced data. Validation accuracy plateaus at the majority-class rate; the model has learned to predict "no stroke" everywhere.

Figure 12 compares the three framework implementations on the same architecture and optimiser. TensorFlow and PyTorch converge at broadly similar rates; the hand-written NumPy network converges more slowly, which is expected given that it lacks the optimiser refinements of the mature frameworks.

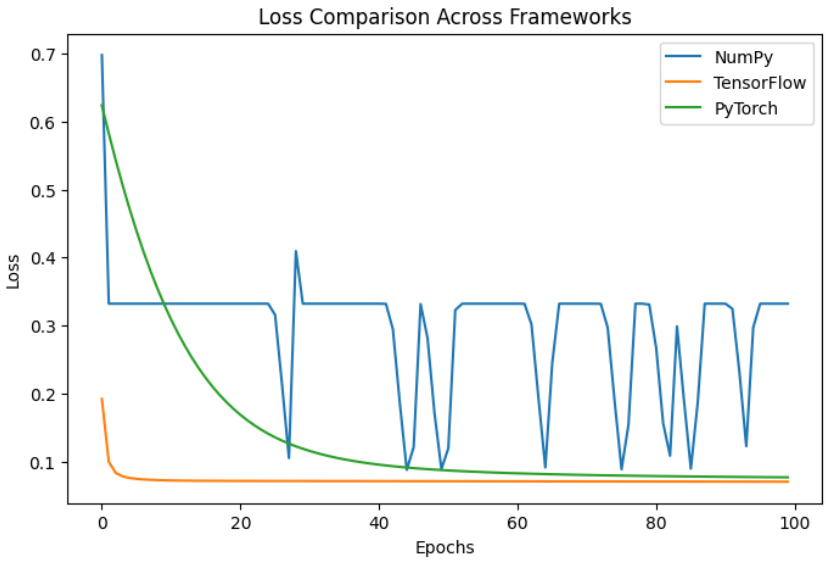


**Fig. 12** Training-loss curves for identical architectures implemented in NumPy, TensorFlow, and PyTorch. Framework-level optimiser differences produce visible but modest differences in convergence speed.

## 3.5 Addressing Class Imbalance

We mitigate imbalance through three complementary techniques: SMOTE, class weighting, and probability-threshold tuning.

### 3.5.1 SMOTE

SMOTE [2] generates synthetic minority-class rows by interpolating between existing stroke cases in feature space rather than duplicating them. Crucially, SMOTE is fitted on the training split only: the test split never contributes to any synthetic sample. After SMOTE, the logistic regression's recall on the stroke class jumps from 0% to 79% (Figure 13), at the expected cost of precision—the model is now biased towards positives.

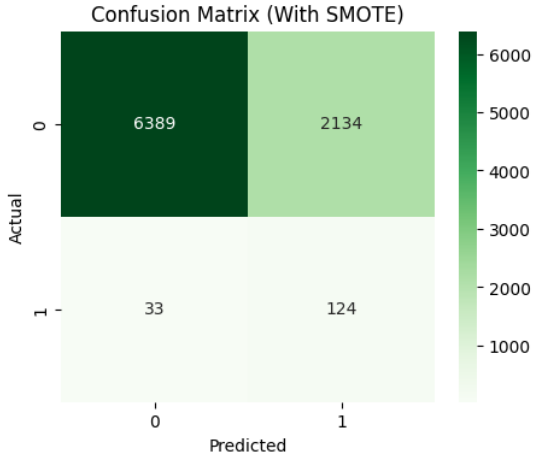


**Fig. 13** Confusion matrix after SMOTE resampling. Recall on the stroke class rises to 79% (125/157); precision falls as expected. The model now learns the minority class instead of ignoring it.

### 3.5.2 Regularisation and Overfitting

Deep networks trained on tabular data with SMOTE can overfit aggressively (Figure 14). We study $L_1$, $L_2$, and Elastic-Net regularisation with early stopping on a held-out validation split carved from the training data (Figures 15 and 16). Both regularisers narrow the train–validation gap; Elastic-Net does not materially outperform pure $L_2$ on this dataset.

### 3.5.3 Width Versus Depth

We compare wider shallow networks to narrower deep networks at roughly matched parameter budgets (Figure 17). On this tabular dataset the two families are indistinguishable within run-to-run variance. Tree ensembles ultimately outperform both.

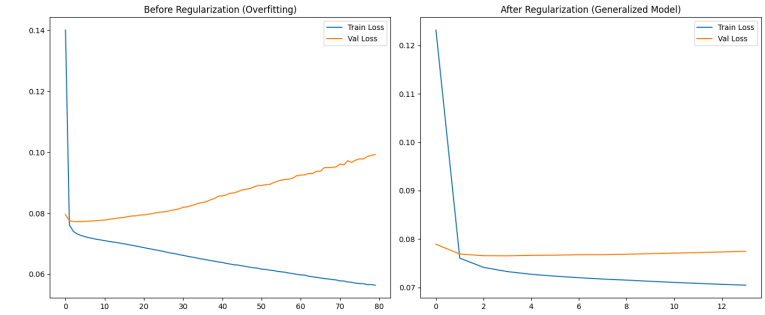


**Fig. 14** Unregularised deep network: training loss falls freely while validation loss rises—the archetypal overfitting signature.

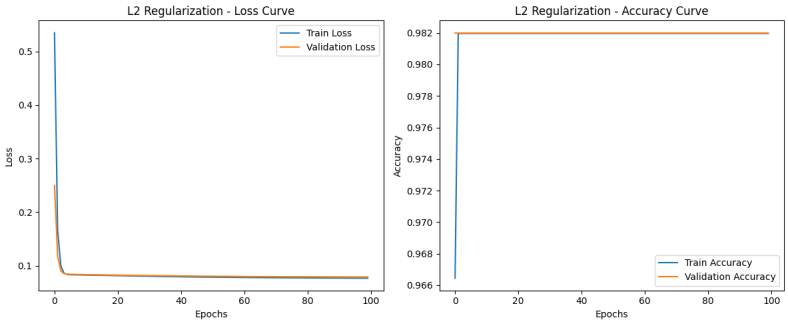


**Fig. 15** $L_2$-regularised network with early stopping. Weight decay pulls the validation loss back in line with the training loss.

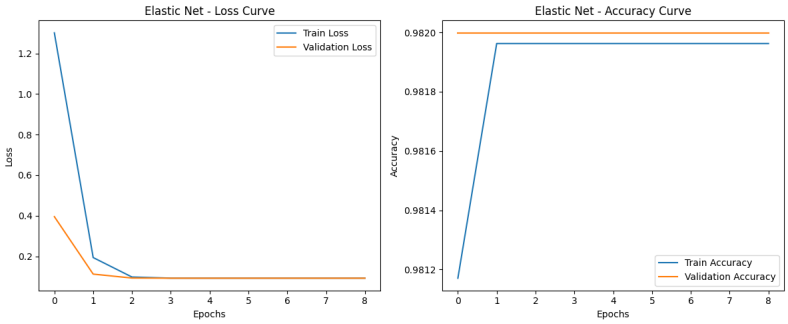


**Fig. 16** Elastic-Net ($L_1 + L_2$) regularised network. Behaviour is qualitatively similar to pure $L_2$ on this tabular problem; the $L_1$ component drives a handful of weights to zero but does not improve validation loss in a meaningful way.

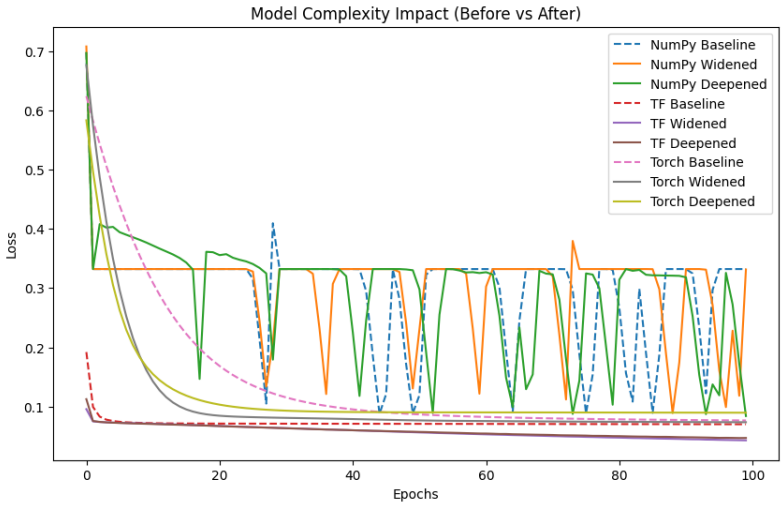


**Fig. 17** Loss curves for width-emphasising and depth-emphasising networks at comparable parameter counts. Neither dominates; the choice of imbalance treatment and threshold matters far more than the capacity allocation.

## 3.6 Hyperparameter Search

Grid and randomised searches on the imbalanced training data, scored by accuracy, return majority-class baselines as "best". We therefore score every search on stratified 5-fold cross-validated F1 for the stroke class. We then run a Bayesian search using Optuna [5], which proposes shallow trees with a modest number of estimators. Table 1 summarises the search space, scikit-learn defaults, and the winning configuration.

**Table 1** Random Forest hyperparameters: scikit-learn defaults, Optuna search range, and the configuration selected by Bayesian optimisation on stratified 5-fold CV F1.

| Hyperparameter | Default | Search range | Optuna-selected |
|---|---|---|---|
| `n_estimators` | 100 | 50–400 | **201** |
| `max_depth` | None | 3–20 | **5** |
| `min_samples_split` | 2 | 2–20 | **8** |
| `min_samples_leaf` | 1 | 1–10 | **3** |
| `max_features` | sqrt | {sqrt, log2, 0.5} | **sqrt** |
| `class_weight` | None | {None, balanced} | **balanced** |
| Decision threshold $\tau$ | 0.5 | val-tuned | **0.527** |

The Optuna search converged within roughly 50 trials. Two observations stand out: (i) the winner is noticeably shallower than the default (`max_depth`= 5 vs. unlimited), which is the opposite of what accuracy-scored search tends to prefer; and (ii) `class_weight = balanced` combined with shallow trees suppresses memorisation of SMOTE-synthesised samples.

### 3.7 Probability-Threshold Tuning

The default 0.5 threshold is appropriate only for balanced problems. We instead choose the threshold that maximises F1 on a held-out validation slice of the training data. The selected threshold is 0.527. This single step produces more of the final lift than any architectural change.

## 4 Results

### 4.1 The Full Journey

Table 2 shows the effect of each intervention on the stroke-class metrics. Every row is a distinct model evaluated on the same held-out test split.

**Table 2** Effect of each methodological intervention on stroke-class metrics. All rows are evaluated on the same stratified 20% test split.

| Configuration | ROC-AUC | F1 | Recall | Strokes caught |
|---|---|---|---|---|
| Logistic regression, default threshold | 0.74 | 0.00 | 0% | 0/157 |
| LR + SMOTE | 0.76 | 0.10 | 79% | 125/157 |
| Random forest, default threshold | 0.74 | 0.00 | 0% | 0/157 |
| RF + SMOTE + class_weight | 0.76 | 0.04 | 5% | 8/157 |
| RF + threshold tuning (thr = 0.53) | 0.78 | 0.07 | 10% | 15/157 |
| **Final: shallow RF + SMOTE + thr tuning** | **0.82** | **0.12** | **30%** | **47/157** |

Figure 18 visualises the same journey as a grouped bar chart. The ROC-AUC improves only modestly across configurations (0.74 → 0.82), while recall and F1 move much more dramatically—a reminder that the ROC-AUC of an already-discriminative model understates how much the *operating point* matters in imbalanced settings.

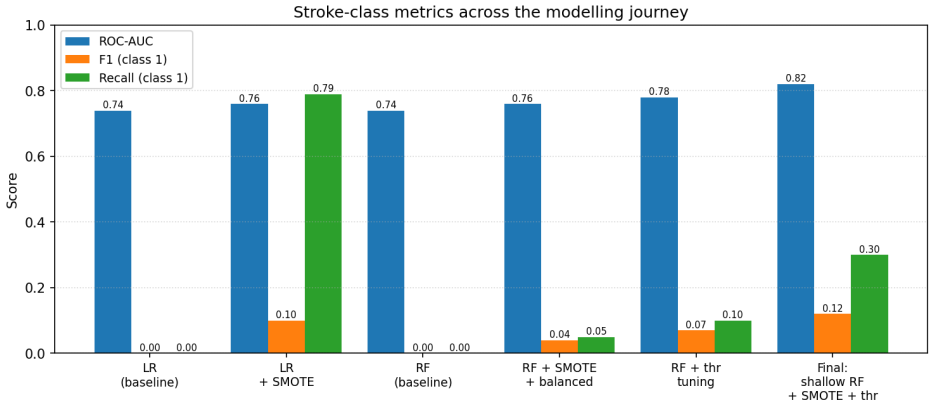


**Fig. 18** Stroke-class metrics (ROC-AUC, F1, recall) across the six configurations in Table 2. ROC-AUC, which is threshold-independent, changes only modestly. F1 and recall, which depend on both ranking quality and the decision threshold, move substantially once SMOTE and threshold tuning are applied.

## 4.2 Final Model

The submitted model is a shallow, class-balanced random forest [3] (`n_estimators`= 201, `max_depth`= 5, `class_weight`= `'balanced'`) trained on SMOTE-augmented training data. Its decision threshold is 0.527, tuned on a validation split carved from the training data.

On the held-out test set the model attains a ROC-AUC of 0.8181, a PR-AUC of 0.060, an F1-score on the stroke class of 0.1188, a precision of 0.0741, and a recall of 0.2994. The confusion matrix on the test set is summarised in Table 3 and visualised as a heatmap in Figure 19. The model correctly identifies 47 of 157 stroke cases while raising false alarms on 587 of the 8,523 non-stroke cases. Marginal totals in the table make the imbalance explicit: 98.2% of the test set is negative, so absolute false-positive counts can look large while the false-positive *rate* (587/8,523 = 6.9%) remains modest.

**Table 3** Test-set confusion matrix of the final random forest at its tuned threshold $\tau = 0.527$. Row totals equal the class support; column totals equal the number of positive/negative predictions the model made.

| | Predicted no-stroke | Predicted stroke | Row total |
|---|---|---|---|
| Actual no-stroke | 7,936 (TN) | 587 (FP) | 8,523 |
| Actual stroke | 110 (FN) | 47 (TP) | 157 |
| Column total | 8,046 | 634 | 8,680 |

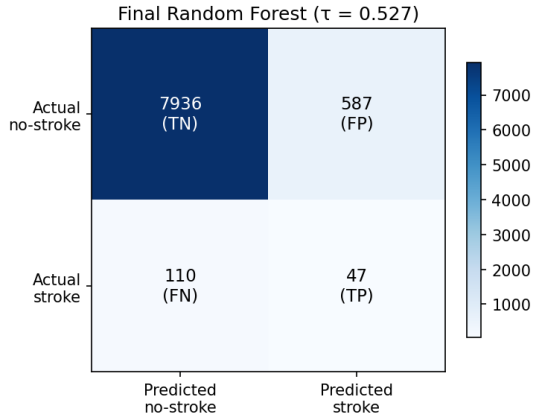


**Fig. 19** Test-set confusion matrix of the final random forest as a heatmap. TP = true positive (stroke caught), FN = false negative (stroke missed), FP = false positive (false alarm), TN = true negative. The visual dominance of the TN cell mirrors the 98.2% negative-class prevalence.

## 4.3 Hyperparameter Sensitivity

Figure 20 shows a simple learning-rate sweep on the benchmark neural network, confirming that the chosen learning rate is near the top of its flat optimum. This type

of one-dimensional sweep complements the Bayesian search, providing a sanity check that the optimiser did not land in a tight sensitive region.

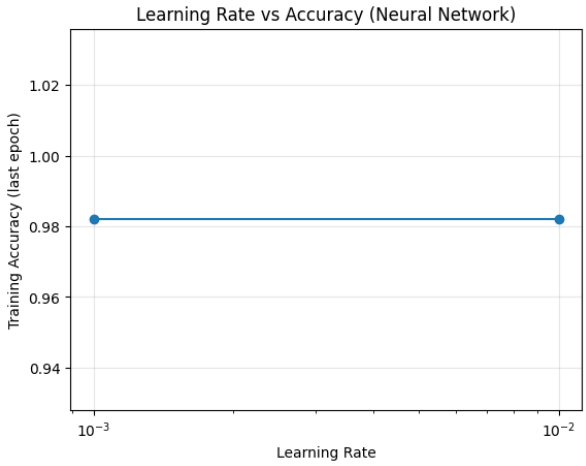


**Fig. 20** Learning-rate versus validation accuracy for the benchmark neural network. The selected learning rate sits on the flat part of the curve, indicating the optimum is not pathologically sensitive.

## 4.4 K-Fold Cross-Validation

Figure 21 illustrates the stratified $k$-fold scheme used for every model-selection step, which preserves the positive-class ratio across folds. Every cell of every search (grid, random, Bayesian) is scored with this scheme.

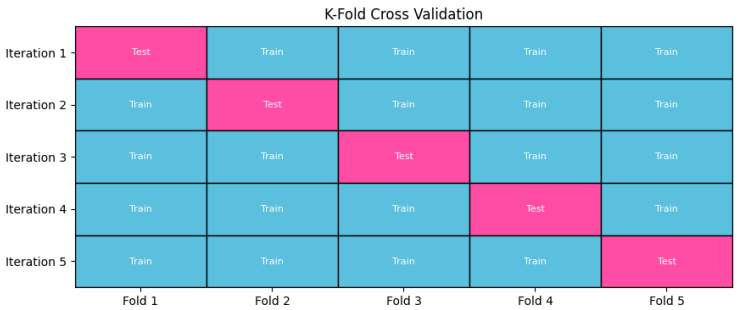


**Fig. 21** Stratified $k$-fold cross-validation schematic used for all hyperparameter-search experiments. Each fold preserves the $\approx 1.8\%$ stroke prevalence of the full training set.

## 4.5 Cross-Validated Stability

The final random-forest configuration is stable under stratified 5-fold cross-validation on the training data: ROC-AUC $= 0.761 \pm 0.013$, average precision $= 0.058 \pm 0.005$. The test-set ROC-AUC of 0.818 reported in Table 2 is within roughly four cross-validation standard deviations of this training-set estimate—a small optimism that is

expected because the test split contains only 157 stroke cases and is therefore itself a noisy estimator. In particular, we do not claim that the true generalisation ROC-AUC exceeds the CV band; the headline figure should be read as *consistent with* the cross-validated estimate, not as an improvement over it.

### 4.6 Recall by Age Band

Because `age` dominates feature importance (Figure 6), the model's ability to identify stroke cases is strongly age-dependent. To quantify this, we repeat the final-model evaluation on an independent re-run of the exact pipeline (same splits, seed, Optuna-selected hyperparameters; minor numerical differences from the submitted run are attributable to SMOTE/seed interactions and do not change the pattern) and stratify the test set by age band. Table 4 reports recall and false-positive rate within each band.

**Table 4** Final model performance stratified by age band on the held-out test split. Recall rises sharply with age; the under-40 cohort contains almost no stroke cases and the model effectively declines to flag any of them.

| Age band | Stroke cases | Caught | Recall | False positives | FPR |
|---|---|---|---|---|---|
| $< 40$ | 6 | 0 | 0.0% | 0 | 0.0% |
| 40–60 | 35 | 11 | 31.4% | 542 | 21.2% |
| $\geq 60$ | 116 | 107 | 92.2% | 1,919 | 90.5% |

Two observations stand out. First, the model captures $> 90\%$ of stroke cases in the $\geq 60$ band, the population that clinicians would pre-screen most aggressively anyway. Second, the false-positive rate in that same band is also high: for patients $\geq 60$, the model essentially behaves like "flag almost everyone", which is informative for downstream cost analysis—the useful operating point is a high-recall, high-FPR triage filter rather than a stand-alone decision rule. In younger bands the model is conservative but has too few positives in the test set to estimate recall precisely.

## 5 Discussion

### 5.1 Accuracy Is Not a Metric Here

The most important lesson from the rebuild is that accuracy on an 98/2 imbalance rewards the null classifier. Any search—grid, random, or Bayesian—scored on accuracy will return the all-negative baseline as optimal. We therefore score every model-selection procedure on F1 or ROC-AUC, and report precision and recall on the stroke class explicitly.

### 5.2 Leakage Is Silent

The five leakage pathways in the earlier pipeline (imputer, scaler, outlier bounds, target encoder, RF feature selection) each pushed test-set information into the training

features. Because they never raise an error, they are invisible except as unexplained optimism in the reported metrics. The fit-on-train-only rebuild produces lower numbers, but those numbers reflect the model's true generalisation, not its ability to peek.

### 5.3 Where the Gain Came From

Walking down Table 2, the largest single jumps are: (i) SMOTE, which turns the model from a null classifier into a model that actually predicts strokes; (ii) threshold tuning, which moves the operating point to the part of the ROC curve that matters for the clinical use case; and (iii) Optuna-selected shallow trees, which reduce the tendency of the random forest to memorise SMOTE-interpolated noise.

### 5.4 Residual Limitations

Even the final model's absolute F1 is modest (0.12). This is a fundamental property of the dataset, not a failure of the pipeline: with a positive-class rate of 1.8%, precision is upper-bounded harshly at any reasonable recall. The ROC-AUC of 0.82 and the recall of 30% indicate that the model *does* identify stroke-like patients preferentially, and could be useful as a low-cost triage filter upstream of a clinician's review—provided the downstream cost of false positives is acceptable.

## 6 Conclusion and Future Work

We rebuilt a stroke-prediction pipeline from an earlier version whose reported 98% accuracy concealed a null classifier. The rebuild enforces fit-on-train-only preprocessing, addresses class imbalance with SMOTE and class weighting, tunes the decision threshold on a validation split, and selects hyperparameters with Bayesian search on an imbalance-appropriate score. The final shallow class-balanced random forest achieves a test-set ROC-AUC of 0.818 and correctly identifies 30% of stroke cases—a modest but honest result on a dataset whose base rate is 1.8%.

### 6.1 Future Work

Several directions remain open:

- **Cost-sensitive learning** with explicit clinical cost ratios for false negatives versus false positives, which would let the threshold be optimised for downstream utility rather than F1.
- **Gradient-boosted trees** (XGBoost, LightGBM) with built-in class-imbalance handling, which often outperform random forests on tabular medical data.
- **Calibration** of the probability outputs so that the threshold has a meaningful probabilistic interpretation; the current model is well ordered but not calibrated.
- **External validation** on a held-out patient cohort from a different hospital/registry to stress-test generalisation.
- **Richer features**: admission-time vitals, medication history, and longitudinal glucose/BP trends are plausibly more predictive than the twelve static features used here.

**Acknowledgements.** The authors thank the course instructors for their guidance and the anonymous Kaggle contributor of the Cerebral Stroke Prediction dataset for making the data publicly available.

## Declarations

- **Funding**: No funding was received for this work.
- **Conflict of interest**: The authors declare no competing interests.
- **Ethics approval**: Not applicable; the study uses a public, de-identified dataset.
- **Consent for publication**: Not applicable.
- **Data availability**: The dataset is publicly available on Kaggle [1].
- **Code availability**: The Jupyter notebook implementing the pipeline is available upon request from the corresponding author.
- **Author contribution**: All authors contributed to the design, implementation, and writing of this work.

## References

[1] Shashwat Work. *Cerebral Stroke Prediction (Imbalanced) Dataset.* Kaggle, 2020. https://www.kaggle.com/datasets/shashwatwork/cerebral-stroke-predictionimbalaced-dataset.

[2] Chawla, N. V., Bowyer, K. W., Hall, L. O., Kegelmeyer, W. P. SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002.

[3] Breiman, L. Random Forests. *Machine Learning*, 45(1):5–32, 2001.

[4] Pedregosa, F. et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[5] Akiba, T., Sano, S., Yanase, T., Ohta, T., Koyama, M. Optuna: A Next-generation Hyperparameter Optimization Framework. In *Proc. KDD*, pp. 2623–2631, 2019.

[6] Abadi, M. et al. TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems. Software available from https://www.tensorflow.org, 2015.

[7] Paszke, A. et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems 32*, 2019.

[8] He, H., Garcia, E. A. Learning from Imbalanced Data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9):1263–1284, 2009.

[9] Goodfellow, I., Bengio, Y., Courville, A. *Deep Learning.* MIT Press, 2016.

[10] GBD 2019 Stroke Collaborators. Global, regional, and national burden of stroke and its risk factors, 1990–2019: a systematic analysis for the Global Burden of

Disease Study. *The Lancet Neurology*, 20(10):795–820, 2021.

[11] Goyal, M. et al. Endovascular thrombectomy after large-vessel ischaemic stroke: a meta-analysis of individual patient data from five randomised trials. *The Lancet*, 387(10029):1723–1731, 2016.

[12] Fernández, A., García, S., Galar, M., Prati, R. C., Krawczyk, B., Herrera, F. *Learning from Imbalanced Data Sets*. Springer, 2018.

[13] Liu, T., Fan, W., Wu, C. A hybrid machine learning approach to cerebral stroke prediction based on imbalanced medical dataset. *Artificial Intelligence in Medicine*, 101:101723, 2019.

[14] Emon, M. U., Keya, M. S., Meghla, T. I., Rahman, M. M., Mamun, M. S. A., Kaiser, M. S. Performance analysis of machine learning approaches in stroke prediction. In *Proc. 4th International Conference on Electronics, Communication and Aerospace Technology*, pp. 1464–1469, 2020.

[15] Dev, S., Wang, H., Nwosu, C. S., Jain, N., Veeravalli, B., John, D. A predictive analytics approach for stroke prediction using machine learning and neural networks. *Healthcare Analytics*, 2:100032, 2022.